

NATURAL LANGUAGE PROCESSING

ANALYSE SÉMANTIQUE ET MODÈLES BASÉS SUR LES TRANSFORMERS (INTUITION)

Sarah Malaeb
2025-26

ANALYSE SÉMANTIQUE EN NLP

- L'analyse sémantique est le processus de compréhension du sens d'un texte dans son contexte.
- Elle va au-delà de la syntaxe (la structure du langage) pour saisir comment les mots, les expressions et les phrases expriment des concepts et comment ils sont reliés entre eux.
- Elle permet de capturer le contexte, l'intention et les relations.

ASPECTS CLÉS DE L'ANALYSE SÉMANTIQUE

Désambiguïsation du sens des mots (Word Sense Disambiguation) : Résolution des ambiguïtés lorsqu'un mot possède plusieurs significations.

Similarité et lien sémantique (Semantic Similarity & Relatedness) : Mesure de la proximité sémantique entre deux mots, phrases ou documents.

Reconnaissance d'entités et de concepts (Entity & Concept Recognition) : Identification des entités (personnes, lieux, compétences) et leur mappage vers des bases de connaissances.

- Exemple : reconnaître « Python » comme langage de programmation, et non comme serpent.

Sens contextuel (Contextual Meaning) : Compréhension que le sens dépend du contexte.

- Exemple : « Apple » = fruit dans un contexte, entreprise dans un autre.

EXEMPLES D'APPLICATIONS DE L'ANALYSE SÉMANTIQUE

- Moteurs de recherche (recherche sémantique plutôt que par mots-clés)
- Systèmes de questions-réponses (réponses contextuelles)
- Systèmes de recommandation (appariement des besoins utilisateurs avec le contenu)
- Cartographie des compétences (compétences / profils de poste)
- Analyse du sentiment et des retours clients

APPROACHES CLASSIQUES

Bag-of-Words (BoW) : Représente le texte par le nombre d'occurrences des mots – ignore l'ordre et le contexte.

TF-IDF (Term Frequency–Inverse Document Frequency) : Pondere les mots selon leur importance dans un document par rapport au corpus.

Latent Semantic Analysis (LSA) : Utilise une factorisation matricielle (SVD) sur TF-IDF pour extraire des thèmes latents. **Latent Dirichlet Allocation (LDA)** : Modélisation probabiliste de sujets.

Limitations:

Représentations éparses

Difficulté à capturer le contexte

Performance limitée pour les tâches à grande échelle.

WORD EMBEDDINGS

Word2Vec : Apprend des vecteurs de mots en se basant sur le contexte environnant.

GloVe (Global Vectors) : Capture les statistiques de co-occurrence des mots.

fastText : Prend en compte les sous-mots et gère les mots rares ou inconnus.

SENTENCE EMBEDDINGS

Moyennage **des embeddings de mots** (simple mais faible : le contexte est ignoré).

Doc2Vec: apprend des embeddings au niveau document (document-level embeddings)

- Étend Word2Vec pour apprendre un vecteur pour l'ensemble du document ou de la phrase en plus des vecteurs mots.

(Introduit un vecteur d'ID de document pendant l'entraînement pour capturer le contexte global.)

- Capture la sémantique au niveau document (pas seulement au niveau mot).
- Conserve les informations contextuelles sur l'utilisation des mots dans un document donné.
- Plus robuste pour les textes longs.
- Toujours limité comparé aux transformers, qui modélisent dynamiquement les dépendances entre mots.

Sentence-BERT (SBERT):

Adapte BERT pour les tâches de similarité sémantique.

Produit des embeddings au niveau phrase optimisés pour la comparaison.

SENTENCE EMBEDDINGS

Exemple:

Sentence A: *“I have experience in cleaning and preparing datasets using Python.”*

Sentence B: *“I built machine learning models for data classification in Python.”*

Averaging embeddings → peut les considérer comme similaires car les deux contiennent « Python » et « data ».

Doc2Vec → légèrement mieux, mais peut encore brouiller la distinction.

BERT/SBERT → identifie correctement que :

La phrase A correspond aux compétences en analyse de données (nettoyage et prétraitement des données).

- La phrase B correspond aux compétences en machine learning (classification, modélisation).

→ **Les modèles sémantiques/contextuels sont essentiels : deux phrases peuvent partager des mots, mais référer en réalité à des blocs de compétences différents.**

DES EMBEDDINGS À LA SIMILARITÉ

Comparer les embeddings en utilisant la similarité cosinus

Exemple (illustrative):

“I love programming in Python” $\leftrightarrow$ “I code in Python” $\rightarrow$ 0.92“

I love programming in Python” $\leftrightarrow$ “I like snakes” $\rightarrow$ 0.15

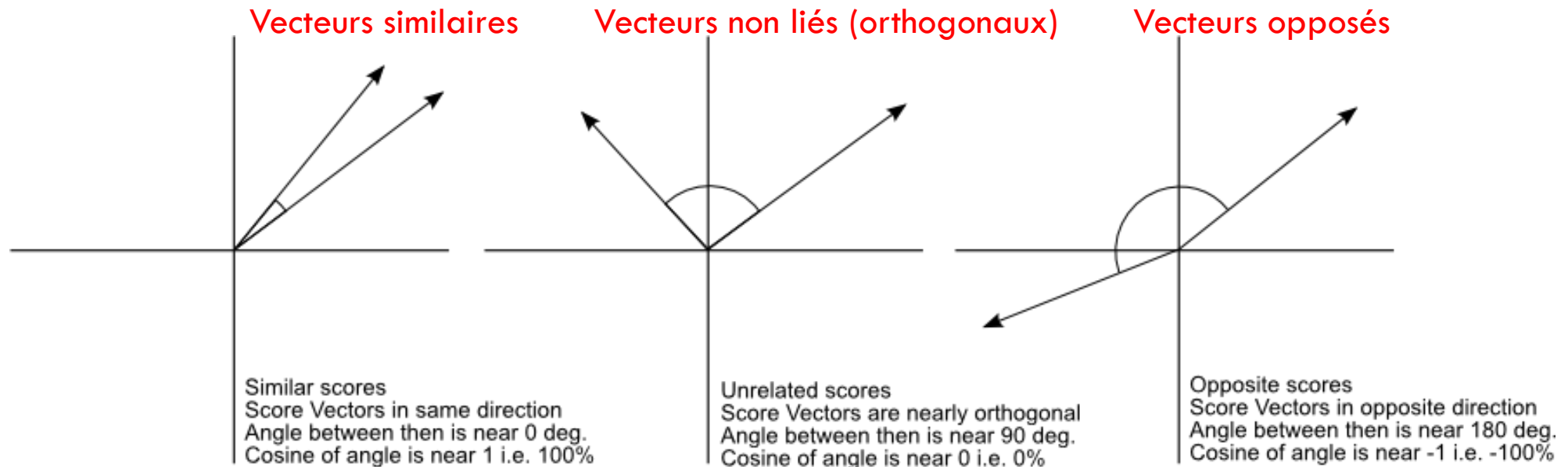
Score élevé $\rightarrow$ forte similarité sémantique

SIMILARITÉ COSINUS

La similarité cosinus est une méthode mathématique permettant de mesurer la proximité sémantique entre deux textes lorsqu'ils sont représentés sous forme de vecteurs.

Elle indique à quel point deux textes sont proches en termes de sens, indépendamment de leur longueur.

Elle est plus efficace que la distance euclidienne en NLP, car **la direction (le sens) importe davantage que la magnitude (le nombre de mots).**



BERT (BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS)

Lit le texte de manière bidirectionnelle (contexte à gauche et à droite).

Tâches de pré-entraînement :

- Masked Language Modeling : masquer certains mots et les prédire, ce qui aide BERT à apprendre des représentations bidirectionnelles et conscientes du contexte.
- Next Sentence Prediction : prédire si une phrase suit logiquement une autre.

Produit des embeddings contextuels pour chaque token.

GPT (GENERATIVE PRETRAINED TRANSFORMER)

Modèle de langage autorégressif.

Prédit le mot suivant en fonction de l'historique.

Conçu pour les tâches de génération.

- Complète BERT (qui est plus adapté à la compréhension).

SBERT (SENTENCE-BERT)

Construit sur BERT, affiné pour la similarité sémantique.

Produit des embeddings au niveau phrase.

Optimisé pour : recherche sémantique, regroupement de phrases, détection de paraphrases...

TRANSFORMERS: UNE REVOLUTION

- Mécanisme d'attention : le modèle apprend sur quels mots se concentrer.
- Traitement parallèle (contrairement aux RNN).
- Encodage positionnel pour capturer l'ordre des mots.
- Architecture encodeur—décodeur.
- Les transformers (BERT, GPT) ont révolutionné le NLP.

SIMILARITÉ SÉMANTIQUE AVEC SBERT

Exemple de code Python :

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = ["I enjoy coding in Python.",
              "Programming in Python is fun!",
              "Snakes live in the jungle."]

embeddings = model.encode(sentences, convert_to_tensor=True)

similarity = util.cos_sim(embeddings[0], embeddings[1])

print("Similarity Score:", similarity.item())
```

PROJECT IA GÉNÉRATIVE

Project à faire dans le cadre du module Projet IA Générative